

Specification Complexity: Reframing Mutation Testing as a Program Complexity Measure

Rohan Vinaik

Abstract

Mutation testing is widely used to evaluate test suite quality by measuring the fraction of syntactic program variants (mutants) that a test suite distinguishes from the original program. We argue that the mutation score is more naturally understood as a measure of *specification completeness* — the degree to which a test suite captures the semantics of the program under test. This reframing gives rise to a natural complexity measure: the *specification complexity* $\sigma(P, \mu)$, defined as the minimum number of tests required to achieve complete specification of a program P under a mutation policy μ . We prove that σ satisfies the Blum axioms for abstract complexity measures — trivially for finite domains, and substantively for partial programs over infinite domains via an obstruction package construction formalized in Lean 4. We establish an exponential separation between program classes under σ , adapting Shannon’s counting argument to the specification setting. We then prove a five-field identification theorem: $\sigma(P, \mu)$ equals the teaching dimension from computational learning theory (Goldman and Mathias, 1996), the query complexity of identity testing from property testing (Blais et al., 2012), the local testability parameter from coding theory, the certificate size from combinatorial complexity, and a new parameter governing membership in a specification-complete complexity class SpecP. While pairwise connections among some of these quantities are known, the identification of all five with a single software engineering quantity appears to be new. We develop a dynamics of specification in which test suite construction follows a greedy trajectory with exponential convergence, Lyapunov stability, and an information-theoretic lower bound of $\log_2(\sigma/(\sigma - 1))$ bits per test. We prove a specification free energy bound showing that greedy specification is variational inference, characterize the dynamic regime transition between polynomial and exponential specification, and establish a composition gap theorem: $\sigma(A \circ B) \leq \sigma(A) + \sigma(B) + \gamma(A, B)$ where γ is bounded by the number of interface mutants and vanishes for independent components. We further develop a compositional specification theory: a sheaf condition under which local specifications compose additively, an obstruction class that measures the cost of violating the sheaf condition, and convergence guarantees for distributed specification under asynchronous communication. The framework provides a candidate formal measure of the gap between statistical and exact learning, connecting to recent arguments (György et al., 2025) that exact learning is essential for general intelligence. All results are formalized in approximately 10,000 lines of Lean 4 with Mathlib; the formalization methodology and tooling are described in Section 6 and Appendix A.

1 Introduction

Abelson and Sussman (1996) open *Structure and Interpretation of Computer Programs* with a deceptively simple claim: programs are “precise specifications of computational processes.” This framing — that a program *is* a specification, not merely something that *has* one — has consequences that the software engineering literature has not fully pursued. If a program is a specification, then the question of what the program computes is a question about the completeness of that specification: how many of the program’s behavioral degrees of freedom are pinned down by external constraints?

Mutation testing provides the measurement mechanism. By generating syntactic variants (mutants) of the program under test and measuring the fraction that a test suite distinguishes

from the original (DeMillo et al., 1978; Hamlet, 1977), mutation testing probes the specification boundary: a surviving mutant is a direction in which the program’s behavior could change without any test noticing. A test suite that distinguishes all non-equivalent mutants is said to achieve a mutation score of 1, typically interpreted as a measure of test suite adequacy.

We propose an alternative interpretation, one that takes Sussman’s framing seriously. A mutation score of 1 means that the test suite *completely specifies* the program’s behavior, up to the distinctions expressible by the chosen mutation policy. Under this reading, the test suite is not merely adequate — it has captured the computational identity of the program in the sense that no syntactically distinct, semantically different program satisfies the same tests. The shift from test adequacy to specification completeness is not merely terminological; it changes which quantities are natural to study and which connections become visible.

Consider what a surviving mutant actually *is* under this reframing. It is not evidence of a lazy tester. It is a syntactic variant of the program that computes a different function, yet no test in the suite distinguishes it from the original. The program could wobble in the direction that mutant represents — could silently compute that other function — and the test suite would not notice. A surviving mutant is, in precise terms, a *behavioral degree of freedom* that the program’s structure leaves unconstrained. The set of all surviving mutants is therefore a map of exactly those dimensions along which the program’s behavior is unspecified. In Sussman’s terms: the program is a specification, but an *incomplete* one — it specifies a computational process whose identity is underdetermined by its test suite.

This reframing converts a quality metric into a structural one. The question changes from “how good is this test suite?” to “how many independent behavioral dimensions does this program expose?” — and the latter is a property of the program, not the tester. A program with many independent behavioral dimensions is hard to specify regardless of who writes the tests; a program whose behavioral dimensions are few and correlated is easy to specify. The difficulty is intrinsic to the program, relative to the mutation policy.

The central quantity of this paper is the *specification complexity* $\sigma(P, \mu)$: the minimum number of tests required to achieve complete specification of a program P under a mutation policy μ . Unlike the mutation score, which is a property of a test suite, $\sigma(P, \mu)$ is a property of the program itself. It measures how many independent constraints are needed to pin down the program’s computational identity — how hard the program is to specify, not how good a particular test suite is.

Scope and model. Throughout, μ denotes a fixed mutation policy; all “equivalences” are with respect to the concept class induced by μ and an oracle model of observability (i.e., tests return exact outputs). The theory is parameterized by μ — different policies yield different values of σ for the same program — and we discuss this dependence explicitly in Section 2.3.

Once specification complexity is understood as a program property, a series of structural consequences follow in rapid succession. The quantity σ turns out to satisfy the Blum axioms for abstract complexity measures, placing it within the framework where speedup theorems, gap theorems, and hierarchy results apply. It admits an exponential separation between program classes — not between programs that are easy and hard to *compute*, but between programs that are easy and hard to *specify*, a distinction that, to our knowledge, has not been formalized. It equals established quantities in five distinct fields of theoretical computer science — pairwise connections among some of these are known, but their simultaneous identification with a concrete software engineering quantity is, to our knowledge, new. And it gives rise to a characteristic dynamics: the process of building a specification has predictable convergence behavior, phase transitions, and information-theoretic optimality guarantees.

Perhaps the most far-reaching consequence is the connection to exact learning. György et al. (2025) recently argued that exact learning — identifying a function precisely, not merely approximately — is the appropriate framework for general intelligence, and that the gap between statistical and exact learning is where the hard problems reside. Specification complexity is,

we show, a formal measure of exactly that gap: the number of constraints required to move from knowing approximately what a program does to knowing exactly. The machinery of this paper — the Blum axioms, the exponential separation, the mutation symmetry group — is, from this vantage point, a theory of how hard exact identification is for particular function classes. This connection was not designed; it was discovered during formalization, and its appearance suggests that specification complexity is measuring something more fundamental than we initially intended.

We establish the following results.

Specification complexity is a complexity measure in the sense of Blum (1967). We prove that σ satisfies both Blum axioms (totality and decidability) for programs over finite domains. This places σ within the framework of abstract complexity theory, where standard hierarchy and gap theorems apply. We further prove that σ is independent of Kolmogorov complexity: there exist programs with low descriptive complexity but high specification complexity, and vice versa.

There is an exponential separation under specification complexity. We construct program classes $\mathcal{C}_{\text{easy}}$ and $\mathcal{C}_{\text{hard}}$ computing Boolean functions on n inputs such that $\sigma(P, \mu) = O(\text{poly}(n))$ for all $P \in \mathcal{C}_{\text{easy}}$ and $\sigma(P, \mu) = \Omega(2^n/n)$ for some $P \in \mathcal{C}_{\text{hard}}$. The argument adapts Shannon’s counting method to the specification setting. This separation is not between programs that are easy and hard to *compute*, but between programs that are easy and hard to *specify* — a distinction that, to our knowledge, has not been formalized.

Specification complexity equals quantities in five distinct fields. We prove that for programs over finite domains with faithful oracles, $\sigma(P, \mu)$ admits exact identifications with: (i) the teaching dimension of P ’s equivalence class in the concept space induced by μ (Goldman and Mathias, 1996); (ii) the query complexity of identity testing (Blais et al., 2012); (iii) the local testability parameter governing local-to-global consistency; (iv) the certificate size for semantic uniqueness; and (v) a new parameter governing membership in the complexity class SpecP, which we introduce as the specification-based analog of P. The proof constructs explicit bijections between the witnessing structures in each field, mediated by a mutation symmetry group G_μ whose algebraic structure determines the computational regime.

Specification has a characteristic dynamics. We prove that greedy test suite construction satisfies a proportional progress guarantee: each test that increases the mutation score does so by at least a $1/\sigma(P, \mu)$ fraction of the remaining gap. This yields exponential convergence to complete specification, a Lyapunov monotonicity result, and an information-theoretic lower bound: each test contributes at least $\log_2(\sigma/(\sigma - 1))$ bits of specification information, and complete specification requires exactly $\log_2(n)$ bits where n is the mutant population size. We further establish a specification free energy bound connecting greedy specification to variational inference, prove that the Regime A/B classification is a dynamic phase transition with a canonical decomposition at the transition point, and prove a composition gap theorem: $\sigma(A \circ B) \leq \sigma(A) + \sigma(B) + \gamma(A, B)$ where γ is bounded by the number of interface mutants and vanishes for independent components.

Compositional specification theory. We prove a sheaf condition under which local specifications compose additively without interface penalties, an obstruction class that measures the total specification cost of violating the sheaf condition, and convergence guarantees for distributed specification under asynchronous communication with partial information sharing. These results extend the composition gap from pairs to arbitrary decomposition lattices and provide the formal bridge from the monolithic theory to practical modular software engineering.

Implementation and methodology. We describe LintGate, a mutation analysis tool designed to operationalize the specification dynamics results, and outline the falsifiable predictions the theory makes for empirical validation. All theoretical results are formalized in Lean 4 with Mathlib across seven phases comprising approximately 10,000 lines. The formalization was produced using an AI theorem prover (Aristotle; Harmonic, 2025) with structural insights and theorem statements originating from the author’s analysis; we discuss the implications of this methodology for mathematical research.

The paper is organized as follows. Section 2 develops the core theory of specification complexity: the definition, basic properties, Blum axiom verification, and the exponential separation. Section 3 presents the dynamics of specification, including free energy bounds, regime transitions, and compositional specification. Section 4 proves the five-field identification theorem and develops mutation symmetry theory. Section 5 develops the compositional specification theory: the sheaf condition, the obstruction class, and distributed convergence. Section 6 describes the LintGate implementation. Section 7 discusses the AI-assisted formalization methodology. Section 8 surveys related work. Section 9 discusses future directions including the SpecP complexity class, topological invariants, and the statistical-to-exact learning gap. A companion monograph provides the complete Lean 4 formalization with detailed commentary.

2 Specification Complexity

The intuition developed in Section 1 — that specification complexity is an intrinsic invariant of a program relative to a mutation policy, measuring how many independent behavioral constraints are needed to pin down a program’s identity — requires a formal foundation before it can bear the weight of theorems. In this section we construct that foundation. We define the mutation system, the specification complexity measure $\sigma(P, \mu)$, and its basic properties, then establish two structural results: that σ is a legitimate complexity measure in the sense of Blum, and that it admits an exponential separation between program classes.

We fix a *mutation system* consisting of a type D (inputs), a type R (outputs), a semantic function $\text{sem} : \text{Program}(D, R) \rightarrow D \rightarrow R$, and a mutation operator μ that generates for each program P a finite set $\text{Mut}_\mu(P)$ of syntactic variants. A test t *kills* a mutant m with respect to P if $\text{sem}(P, t.\text{input}) \neq \text{sem}(m, t.\text{input})$. Two programs are *truly equivalent*, written $P_1 \equiv P_2$, if $\text{sem}(P_1, x) = \text{sem}(P_2, x)$ for all $x \in D$. The *non-equivalent mutants* of P are $\text{Mut}_\mu^\neq(P) = \{m \in \text{Mut}_\mu(P) : m \not\equiv P\}$.

A test suite T *achieves specification completeness* ($\text{SC} = 1$) for P under μ if T kills every non-equivalent mutant.

Definition 2.1 (Specification complexity). The specification complexity of a program P under mutation policy μ is

$$\sigma(P, \mu) = \min\{|T| : T \text{ achieves } \text{SC} = 1 \text{ for } P \text{ under } \mu\}.$$

In the Lean formalization, this corresponds to:

```
def specComplexity (sem : Program D R → D → R)
  (oracle : Test D R → R → Bool)
  (MS : MutationSystem D R) (P : Program D R) : ℕ :=
  sInf { k | ∃ T : TestSuite D R, T.card = k ∧ achievesFullSC' sem oracle MS P T }
```

The quantity $\sigma(P, \mu)$ depends on both the program and the mutation policy. A coarser policy (fewer mutants) yields lower specification complexity; a finer policy yields higher complexity. This dependence is analogous to the dependence of circuit complexity on the gate basis: the choice of primitives affects the measure, but the structural theory transfers across choices. The key point is that once the mutation policy is fixed, σ is determined entirely by the program — it is a property of the computational object, not of any particular attempt to test it. If programs are specifications (Abelson and Sussman, 1996), then σ measures how many independent observations are needed to determine which specification a program embodies — how many constraints are needed to disambiguate the program’s computational identity from its syntactic neighbors. This is the sense in which specification complexity is a *complexity measure* rather than a *quality metric*: it characterizes the inherent difficulty of the specification task, not the skill of the person performing it.

Theorem 2.2 (Finiteness). *For finite domains with decidable equality, $\sigma(P, \mu) \leq |\text{Mut}_\mu^\neq(P)|$. One test per non-equivalent mutant always suffices.*

Proof. For each $m \in \text{Mut}_\mu^\neq(P)$, since $m \neq P$, there exists an input x with $\text{sem}(P, x) \neq \text{sem}(m, x)$. A test on this input kills m . The collection of such tests has cardinality at most $|\text{Mut}_\mu^\neq(P)|$. (Lean: `T5_1_spec_complexity_finite`.) $\square$

$\square$

Theorem 2.3 (Representation independence). *If $P_1 \equiv P_2$ and there exists a mutation-preserving map $\phi : \text{Mut}_\mu(P_1) \rightarrow \text{Mut}_\mu(P_2)$ such that $m \equiv \phi(m)$ for all $m \in \text{Mut}_\mu(P_1)$, then $\sigma(P_1, \mu) \leq \sigma(P_2, \mu)$. (Lean: `T5_2_representation_bound`.)*

This establishes that σ is a property of the computation rather than the syntax, up to the structure of the mutation operator. Refactoring a program does not change its specification complexity, provided the refactoring preserves the mutation structure — a reassuring property for a measure intended to characterize programs rather than their notation.

Theorem 2.4 (Independence from Kolmogorov complexity). *Specification complexity and Kolmogorov complexity are independent: there exist programs with $K(P) = O(\log n)$ and $\sigma(P, \mu) = \Omega(2^n/n)$, and programs with $K(P) = \Omega(2^n)$ and $\sigma(P, \mu) \leq 1$. (Lean: `T5_3_conditional_refactor`.)*

The first witness is a program that computes a pseudorandom function: short to describe, but its apparent randomness makes it expensive to distinguish from mutants. The second is a lookup table encoding a function with a single distinguishing input: long to write down, but one test suffices to identify it. Specification complexity thus measures something fundamentally different from descriptive complexity — not how much information a program *contains*, but how much of that information is *externally observable* through testing. A program can be simple to describe yet hard to specify (because its compact description generates complex, entangled behavior), or complex to describe yet easy to specify (because its sprawling implementation computes something with few behavioral degrees of freedom). The two notions of complexity are, in a precise sense, orthogonal.

2.1 Blum axiom satisfaction

Blum (1967) defined an abstract complexity measure as a function satisfying two axioms: totality (the measure is defined on every input) and decidability (given an input and a bound, it is decidable whether the measure exceeds the bound). We show that σ satisfies both.

Theorem 2.5 (Blum axioms). *For mutation systems over finite domains: (i) $\sigma(P, \mu)$ is defined for every program P ; (ii) given P and $k \in \mathbb{N}$, it is decidable whether $\sigma(P, \mu) \leq k$. (Lean: `T5_7b_decidable_finite_domain`, `T5_8_blum_axiom_1`.)*

For finite domains (Option A in the formalization), axiom satisfaction is straightforward: totality follows from finiteness, and decidability holds because one can enumerate all test suites of size k and check whether any achieves $\text{SC} = 1$. The procedure is exponential in k , but decidability requires only existence, not efficiency. We note this candidly: any function from finite sets to $\mathbb{N}$ with a decidable graph trivially satisfies weak versions of Blum’s axioms.

The substantive case (Option B in the formalization) extends σ to partial programs over infinite domains. Here the codomain is modeled as $\mathbb{N}_\infty$ (the one-point compactification of $\mathbb{N}$, representing possible non-termination), and the forward direction of the decidability proof requires constructing an *obstruction package* — a finite certificate witnessing that no test suite of size $\leq k$ achieves $\text{SC} = 1$. The obstruction package isolates the combinatorial content of the undecidability barrier and shows it can be finitely certified even when the domain is infinite. This construction is formalized in Lean 4 (`T5_8b_ForwardObstructionPackage`) and is, in our view, where the Blum axiom verification becomes genuinely nontrivial.

Blum axiom satisfaction — in either case — places σ within the framework of abstract complexity theory, and this placement has consequences. The standard hierarchy machinery — speedup theorems, gap theorems, the existence of arbitrarily hard instances — all apply. But the conceptual significance runs deeper than the technical inheritance. Blum’s axioms were designed to capture what it means for a quantity to measure computational difficulty in a resource-independent way. The fact that σ satisfies them means that “how hard is this program to specify?” is a well-posed complexity-theoretic question in the same sense as “how long does this program take to run?” or “how much memory does it require?” Specification difficulty is, in this formal sense, a well-posed complexity-theoretic question alongside time and space.

2.2 Exponential separation

The main complexity-theoretic result of this section is a separation between program classes under σ .

Theorem 2.6 (VCC exponential separation). There exist classes $\mathcal{C}_{\text{easy}}$ and $\mathcal{C}_{\text{hard}}$ of programs computing Boolean functions on n inputs such that $\sigma(P, \mu) = O(\text{poly}(n))$ for all $P \in \mathcal{C}_{\text{easy}}$ and $\sigma(P, \mu) = \Omega(2^n/n)$ for some $P \in \mathcal{C}_{\text{hard}}$. (*Lean: T5_11_Verification_Can_Be_Exponentially_Easier_Than_Computation.*)

The lower bound follows a Shannon-style counting argument: the number of distinct Boolean functions on n inputs is 2^{2^n} , while a test suite of size k can distinguish at most 2^k equivalence classes among the mutants of any given program. For the lower bound to fail, we would need $2^k \geq 2^{2^n}/\text{poly}(n)$, requiring $k = \Omega(2^n/n)$. The upper bound arises from programs with decomposable structure whose mutant spaces admit logarithmic-size distinguishing sets: programs whose behavioral dimensions are independent can be specified one dimension at a time.

The lower bound argument is Shannon’s counting method (Shannon, 1949), applied to specification rather than computation. The proof technique is standard; the contribution is showing that it applies in this setting and yields a clean result. The conceptual content, however, is distinct from Shannon’s original: his result says that most functions are hard to *compute*; ours says that most functions are hard to *specify*. These are different claims — Theorem 2.4 shows that computational and specification complexity are independent — and their conjunction paints a richer picture of what makes programs difficult. A program can be easy to compute but hard to specify (a pseudorandom function), easy to specify but hard to compute (a function with a single distinguishing input but no efficient algorithm), or hard in both senses. The exponential separation provides a formal justification for the empirical observation that some programs are much harder to test than others — not because the tester is insufficiently clever, but because the program’s specification complexity is inherently high.

2.3 The role of the mutation policy

A candid assessment of the framework requires confronting the elephant in the definition: $\sigma(P, \mu)$ depends on μ , and different mutation policies can yield different specification complexities for the same program. This is not a defect — it is an inherent feature of any complexity measure defined relative to a class of perturbations — but it demands more discussion than a passing analogy to gate bases in circuit complexity. (The analogy is imperfect: standard gate bases are polynomially related — any circuit over one basis can be simulated with polynomial overhead in another, a consequence of the universality of NAND and related results — but no analogous robustness theorem exists for mutation policies. Proving or disproving such a result is the central open problem of the framework.)

Three natural mutation policies for programs computing functions $f : D \rightarrow R$ illustrate the range. *Point mutations* (flipping one output value) yield $\sigma = |\{x : f \text{ is not locally constant at } x\}|$, which is essentially the number of non-redundant inputs. *Operator mutations* (replacing arithmetic or logical operators in an AST) yield σ that depends on the syntactic structure — two programs computing the same function may have different σ if their ASTs differ, making this policy representation-dependent in a way that point mutations are not. *Block mutations* (replacing entire subexpressions) yield σ that tracks the compositional structure of the program, and is the policy most closely related to the decomposition prescriptions of Section 6.

The dependence on μ has two consequences. First, *all results in this paper that mention σ are implicitly parameterized by μ* — the Blum axioms, the separation, the five-field identification all hold for any fixed mutation policy satisfying mild finiteness conditions. The theory is uniform in μ ; only the numerical values change. Second, *the choice of μ determines what “specification” means* — it defines the resolution at which the program’s behavior is probed. A coarse policy asks “does this program compute roughly the right function?” while a fine policy asks “does it compute exactly this function, implemented in exactly this way?” Neither question is more natural than the other; they serve different purposes, just as different gate bases serve different purposes in circuit complexity.

What we do *not* provide is a canonical choice of μ . This is the principal limitation of the current framework and the most important direction for future work. A satisfactory theory of mutation policies would include: a taxonomy of natural policies, their relationships to each other, conditions under which σ is invariant across a class of policies, and empirical guidance on which policies yield the most useful decomposition signals for practical software engineering. We regard the absence of this theory as an honest gap, not a fatal one — circuit complexity thrived for decades without a canonical gate basis, and the structural theory (Sections 3–5) is rich enough to motivate the search.

2.4 Teaching dimension embedding

The first connection to structures outside software engineering comes from computational learning theory.

Theorem 2.7 (Teaching dimension embedding). For programs over finite domains with faithful oracles, $\sigma(P, \mu)$ equals the teaching dimension of P ’s semantic equivalence class in the concept space induced by μ . (*Lean: `T5_17_teaching_dimension_lower_bound`, `T5_17_teaching_dimension_upper_bound`.*)

The teaching dimension (Goldman and Mathias, 1996) is the minimum number of labeled examples needed to uniquely identify a target concept in a concept class. A test that kills a mutant m is a labeled example distinguishing the target from m ; the minimum test suite achieving $\text{SC} = 1$ is precisely a minimum teaching set. This identification is not merely analogical — the witnessing structures are identical. This is the first of five such identifications; Section 4 develops the remaining four. The breadth of these connections suggests that specification

complexity is not an artifact of our particular formalization, but a natural quantity that different fields have been studying under different names.

3 Specification Dynamics

The results of Section 2 answer a static question: how many tests does a program require? But specification is not an instantaneous event — it is a process that unfolds over time, and the *trajectory* of that process carries information that the endpoint alone does not reveal. Two programs with identical specification complexity $\sigma(P, \mu) = k$ may have radically different specification trajectories: one converges smoothly as each test eliminates a predictable fraction of surviving mutants; the other exhibits erratic progress, with long plateaus followed by sudden collapses, as entangled behavioral dimensions resist independent elimination. The trajectory is a diagnostic signature of the program’s internal structure — a time series that reveals, step by step, where the behavioral complexity is concentrated and how the program’s degrees of freedom interact.

This section develops the dynamics of specification. We prove that the greedy specification process exhibits exponential convergence, a phase transition between qualitatively different testing regimes, Lyapunov stability, and information-theoretic optimality guarantees. These are not merely bounds on an algorithm; they are structural properties of the specification process itself, and they connect the abstract complexity measure σ to observable phenomena in testing practice.

3.1 Specification trajectories

A *specification trajectory* is a sequence of test suites $T_0 \subset T_1 \subset \dots \subset T_n$ where $T_0 = \emptyset$ and T_n achieves $\text{SC} = 1$. Formally, it is a monotone chain in the lattice of test suites ordered by inclusion, terminating at complete specification. The killed set $K_i = \text{killed}(T_i)$ is monotonically nondecreasing and the survivor set $S_i = \text{Mut}_\mu^\neq(P) \setminus K_i$ is monotonically nonincreasing, with $S_n = \emptyset$. (Lean: `SpecificationTrajectory`, `killed_trajectory_mono`, `survivor_set_empty_iff_killed_eq_all`.)

The kill increment $\Delta K_i = |K_{i+1}| - |K_i|$ measures how many new mutants are killed at step i , and the specification score $\text{SC}_i = |K_i|/|\text{Mut}_\mu^\neq(P)|$ gives the fraction of non-equivalent mutants killed so far. Every trajectory begins at $\text{SC}_0 = 0$ and ends at $\text{SC}_n = 1$; the question is how the intermediate values behave.

3.2 Greedy convergence

The *greedy algorithm* at each step selects the test that kills the most surviving mutants. The central convergence result is that this algorithm makes proportional progress at every step.

Theorem 3.1 (Proportional progress). Let T^* be an optimal test suite with $|T^*| = \sigma(P, \mu)$. At any step of the greedy algorithm with survivor set $S_i \neq \emptyset$, there exists a test killing at least $|S_i|/\sigma(P, \mu)$ survivors. (Lean: `greedy_proportional_progress`.)

Proof sketch. Since T^* kills all non-equivalent mutants, by a pigeonhole argument over the tests in T^* , at least one test in T^* kills at least $|S_i|/|T^*|$ of the current survivors. The greedy choice is at least as good. (The full proof in Lean is a careful application of `Finset.card_biUnion_le` with a contrapositive argument.) $\square$

$\square$

We note that the proportional progress guarantee and its consequences (Theorems 3.2–3.3) follow from the standard analysis of the greedy algorithm for set cover (Chvátal, 1979). The contribution is not the proof technique but the application domain: that specification trajectories *are* set cover instances, with the consequence that the substantial existing theory of set cover applies directly to test suite construction.

The proportional progress guarantee has an immediate dynamical consequence.

Theorem 3.2 (Exponential decay). Under the greedy algorithm, the survivor count satisfies $|S_i| \leq |S_0| \cdot (1 - 1/\sigma(P, \mu))^i$. All mutants are killed after at most $\sigma(P, \mu) \cdot \ln |S_0|$ steps. (*Lean: exponential_decay_of_survivors.*)

The proof uses the standard inequality $(1 - 1/k)^i \leq e^{-i/k}$ and the observation that $|S_0| \cdot e^{-i/k} < 1$ implies $|S_i| = 0$ since survivor counts are integers. The convergence rate $1/\sigma(P, \mu)$ connects the dynamics to the complexity measure: programs with low σ converge fast; programs with high σ converge slowly. This is not merely a bound — it is the characteristic timescale of specification.

Theorem 3.3 (Greedy approximation ratio). The greedy algorithm uses at most $\sigma(P, \mu) \cdot (H(|S_0|))$ tests, where H is the harmonic number. This is optimal within an $O(\ln n)$ factor. (*Lean: T5_12_greedy_approximation_ratio.*)

The $O(\ln n)$ gap is inherited from the greedy algorithm for set cover (Chvátal, 1979), of which specification is a special case. Whether the gap can be closed for specification specifically — whether the structure of mutation systems admits better algorithms than generic set cover — is an open question.

3.3 Phase transition

The dynamics of specification exhibit a qualitative transition between two regimes.

Theorem 3.4 (Bulk-to-tail transition). There exists a critical step i^* such that for all $i < i^*$, the kill increment satisfies $\Delta K_i \geq 2$. Moreover, $i^* \leq |S_0| - \sigma(P, \mu)$, provided $|S_0| \geq 2\sigma(P, \mu)$. (*Lean: phase_transition, transition_step_properties.*)

Before the transition, mutants die in correlated clusters: each test simultaneously eliminates multiple behavioral variants, because many mutants share the same distinguishing input. After the transition, each surviving mutant requires its own targeted test. The bulk phase is efficient (many mutants per test); the tail phase is expensive (one mutant per test at worst). The transition point i^* is observable in practice and marks the moment when further testing yields diminishing returns — or, equivalently, the moment when the remaining specification complexity concentrates in independent behavioral dimensions.

The practical implication is that the phase transition creates a structural prescription for software design. A function whose specification trajectory enters the tail phase early has tangled behavioral dimensions — its behavioral degrees of freedom are not independent, so tests cannot eliminate them in correlated clusters. Extracting those dimensions into separate functions can reduce each component’s σ even if the total number of mutants is unchanged. The specification trajectory thus generates a decomposition signal: not a stylistic preference for smaller functions, but a complexity-theoretic argument that entangled behavioral dimensions are *inherently more expensive to specify* than independent ones.

This prescription connects directly to Sussman’s concept of *abstraction barriers* (Abelson and Sussman, 1996). When a function has surviving mutants across multiple categories — arithmetic, conditional, and string mutations all surviving simultaneously — it has violated an abstraction barrier by mixing concerns that should be separated. The surviving mutation categories are, in effect, a constructive diagnostic of which abstraction barriers are missing. The

operational chain is: mutation pressure reveals specification gaps; specification gaps identify entangled concerns (violated abstraction barriers); decomposition along those concerns produces components whose behavioral dimensions are independently testable; and independently testable components are, by construction, the ones whose specification complexity is additive rather than multiplicative.

Viewed through the exact learning lens (Section 4.3), the phase transition has a precise interpretation. In the bulk phase, each test eliminates correlated mutants — it is performing statistical learning, because each observation constrains many alternatives simultaneously. In the tail phase, each surviving mutant is effectively independent — the tester has exhausted the correlations and must resort to exact identification of each remaining behavioral degree of freedom individually. The transition point i^* is thus the moment when the problem shifts from statistical to exact learning, and the specification complexity σ measures how many degrees of freedom must be resolved in the exact regime. This connection provides a formal basis for the empirical observation that the “last few mutants” are disproportionately expensive to kill: they are expensive not because the tester lacks ingenuity, but because the problem has changed character.

3.4 Trajectory stability

The *trajectory stability* measures the average change in kill rate across consecutive steps:

$$\text{Stab}(S, n) = \frac{1}{n} \sum_{i=0}^{n-1} |\Delta K_{i+1} - \Delta K_i|.$$

Theorem 3.5 (Stability bound). For a greedy trajectory, $\text{Stab}(S, n) \leq (|S_0|/n) \cdot (1 + 1/\sigma(P, \mu))$. (Lean: `greedy_stability_bound_final`.)

The proof depends on a structural result about greedy traces: the kill increments are monotonically nonincreasing (Lean: `greedy_trace_implies_mono`), so the sum of absolute differences telescopes. Smooth convergence (low stability) indicates a well-structured program whose behavioral dimensions are approximately independent. Erratic convergence (high stability) indicates entangled concerns.

Remark. The survivor count serves as a Lyapunov function for the specification process: it is nonnegative, monotonically nonincreasing along trajectories, and zero if and only if specification is complete (Lean: `lyapunov_monotonicity`). We state this as a remark rather than a theorem because it follows directly from the definitions; its value is terminological, connecting the specification dynamics to the standard stability framework rather than establishing a new result.

3.5 Information-theoretic analysis

The specification entropy at step i is $H_i = \log_2 |S_i|$, measuring the remaining uncertainty about the program’s identity among its mutant neighborhood.

Theorem 3.6 (Entropy monotonicity). H_i is monotonically nonincreasing along any specification trajectory. For the greedy algorithm, $H_0 = \log_2 n$ and $H_n = 0$. (Lean: `entropy_monotonicity`, `entropy_properties`.)

Theorem 3.7 (Information per test). Each greedy test contributes at least $\log_2(\sigma(P, \mu)/(\sigma(P, \mu) - 1))$ bits of specification information. The total information required for complete specification is exactly $\log_2 n$ bits, where n is the number of non-equivalent mutants. (Lean: `greedy_information_content`, `total_information_content`.)

The total information result follows from a telescoping argument: the sum $\sum_i \log_2(|S_i|/|S_{i+1}|)$ collapses to $\log_2(|S_0|/|S_n|) = \log_2 n$. The per-test lower bound shows that the greedy algorithm extracts specification information at a rate that is optimal to within a logarithmic factor, consistent with the set cover approximation ratio of Theorem 3.3.

3.6 Greedy versus random testing

Theorem 3.8 (Trajectory comparison). Let T^G and T^R denote greedy and random trajectories respectively. The greedy trajectory satisfies $|T^G| \leq \sigma(P, \mu) \cdot H(|S_0|)$. The random trajectory satisfies $|T^R| \geq \lceil 1/(2\delta_{\min}) \rceil$ with probability at least $3/4$, where $\delta_{\min}$ is the minimum disagreement probability over all non-equivalent mutants. (*Lean: T7_7_trajectory_comparison.*)

The gap between greedy and random testing depends on the mutation system’s structure. In Regime A (local mutations with large symmetry groups), $\delta_{\min}$ is bounded below and random testing is polynomially competitive with greedy. In Regime B (global mutations with small symmetry groups), $\delta_{\min}$ can be exponentially small, making random testing exponentially worse. This regime analysis — developed fully in Section 4 — explains why random testing works well for some programs and catastrophically fails for others.

3.7 Specification free energy and redundancy

The convergence results of Sections 3.2–3.4 describe the *rate* of specification. The following results characterize the *thermodynamic* structure: greedy specification minimizes a free energy functional, and it does so without waste.

Definition 3.9 (Specification free energy). The specification free energy at step i is

$$F_i = H_i + \sigma \cdot r_i$$

where $H_i = \log_2 |S_i|$ is the specification entropy and r_i is the number of tests remaining. The free energy combines the entropic cost (remaining uncertainty) with the specification cost (remaining test budget).

Theorem 3.10 (Specification free energy bound). The greedy algorithm satisfies $F_{i+1} \leq F_i - \log_2(\sigma/(\sigma - 1))$: each test step decreases the free energy by at least the information-theoretic minimum. (*Lean: specificationFreeEnergyBound.*)

The free energy formulation reveals greedy specification as variational inference: the algorithm minimizes a free energy functional over the space of test suites, with each step providing a guaranteed descent. This connects the convergence analysis of Theorem 3.2 to the statistical physics of learning, and provides a dual perspective on the information bound of Theorem 3.8. The free energy is monotonically non-increasing along greedy trajectories; the descent rate $\log_2(\sigma/(\sigma - 1))$ is a universal constant determined entirely by the specification complexity.

Theorem 3.11 (Redundancy characterization). A test is specification-redundant if and only if it contributes zero information gain — it kills no surviving mutant not already killed by the existing suite. The greedy algorithm has redundancy ratio $\rho = 0$: every selected test is specification-effective. Moreover, for any trajectory achieving full specification, the redundancy ratio converges to 1 in the limit as additional tests are appended. (*Lean: redundantIffZeroInformation, greedy_redundancy_ratio_zero, redundancy_ratio_limit_eq_one.*)

Theorem 3.12 provides a formal justification for the greedy strategy beyond convergence rate: not only does it converge exponentially (Theorem 3.2), it does so without waste. Every test selected by the greedy algorithm contributes positively to specification — a property that

random testing conspicuously lacks. The asymptotic redundancy ratio of 1 confirms the intuition that once full specification is achieved, *all* further tests are redundant — the program’s behavioral identity has been completely determined, and additional observations provide no new information.

3.8 Regime dynamics

The regime analysis of Section 3.6 is static: it classifies programs into Regime A (polynomial σ , efficient random testing) and Regime B (exponential σ , random testing fails) but does not describe how the regime evolves *during* specification. The following results characterize the dynamic structure.

Theorem 3.12 (Spectrum evolution). The local disagreement spectrum Λ_i — the multiset of per-mutant disagreement probabilities at step i — has nonincreasing size along any specification trajectory. Moreover, $\max(\Lambda_i)$ is antitone and $\min(\Lambda_i)$ is monotone. (*Lean: `spectrumSizeAntitone`, `spectrum_max_antitone`, `spectrum_min_monotone`.*)

The spectrum encodes the difficulty profile of remaining mutants. As specification proceeds, easy mutants are eliminated first (decreasing the maximum) while the minimum disagreement probability increases — the residual mutants become uniformly harder. This spectral compression explains the empirical observation that the tail phase of mutation testing is disproportionately expensive.

Theorem 3.13 (Dynamic regime transition). For any specification trajectory, there exists a well-defined transition point i_{AB} such that the system operates in Regime A for $i < i_{AB}$ and in Regime B for $i \geq i_{AB}$. The transition point is determined by the symmetry structure of the surviving mutant population. (*Lean: `dynamicRegimeTransition`.*)

Theorem 3.14 (Regime decomposition). At the transition point i_{AB} , the surviving mutant population admits a canonical decomposition into an easy component M_{easy} (supporting Regime A — any nonempty subset has a test killing ≥ 2 mutants) and a hard component M_{hard} (forcing Regime B — every test kills ≤ 1 mutant). The components are separated: no test crosses both. The transition index coincides with the exhaustion of M_{easy} , and all sub-blocks of M_{easy} preserve Regime-A support. (*Lean: `regimeTransition`.*)

Together, Theorems 3.14 and 3.15 establish that the phase transition of Theorem 3.4 is not merely a change in convergence rate but a structural transition between qualitatively different specification regimes. The decomposition at i_{AB} is canonical — it is determined by the mutation system, not by the choice of testing strategy. The Regime-A support invariant under subsets (*Lean: `supports_Regime_A_mono`*) means that the decomposition is robust: refining the partition preserves the structural classification.

3.9 Compositional specification

The preceding results treat programs monolithically. For software engineering, where programs are composed from components, the central question is: how does specification complexity behave under composition?

Theorem 3.15 (Composition gap). For programs A and B with composition $A \circ B$, the specification complexity satisfies

$$\sigma(A \circ B, \mu) \leq \sigma(A, \mu) + \sigma(B, \mu) + \gamma(A, B)$$

where $\gamma(A, B) \geq 0$ is the composition gap. If A and B are specification-independent — no mutant of A is distinguishable only via interaction with B , and vice versa — then $\gamma(A, B) = 0$ and specification complexity is additive. (*Lean: `compositionGap`, `gammaNonneg`, `gammaZeroIfIndependent`.*)

Theorem 3.16 (Interface mutant bound). The composition gap is bounded by the number of interface mutants: $\gamma(A, B) \leq |\text{InterfaceMutants}(A, B)|$, where an interface mutant is one that is neither expressible as $m_A \circ B$ (a mutation in A ’s domain) nor as $A \circ m_B$ (a mutation in B ’s domain) — its distinguishability depends intrinsically on the interaction between A and B . (Lean: `gammaLeInterfaceMutantsCard`.)

The composition complexity decomposes into three layers: A ’s internal complexity ($\sigma(A)$ tests distinguish A ’s mutants), B ’s internal complexity ($\sigma(B)$ tests distinguish B ’s mutants), and an interface obstruction (γ tests needed for mutants at the composition boundary). The proof uses surjectivity and injectivity properties of the component projections to justify that composed test suites can be constructed from component test suites with bounded overhead (Lean: `kappa_comp_right_of_surjective`, `kappa_comp_left_of_injective`).

Theorems 3.16 and 3.17 provide the formal bridge from the monolithic theory to the decomposition practice described in Section 6. The composition gap γ measures exactly the specification cost of component interaction. When components are well-separated (small interface), γ is small and the specification effort for the composition is close to the sum of the parts. When components are tightly coupled (large interface), γ can dominate, explaining why monolithic programs are disproportionately expensive to specify. The bound $\gamma \leq |\text{InterfaceMutants}|$ is computable: LintGate can estimate the composition gap directly from the mutation analysis, providing a quantitative guide for decomposition decisions.

4 The Five-Field Identification

The most striking feature of specification complexity is not any single theorem about it, but the breadth of its connections to established theory. Five distinct fields of theoretical computer science — computational learning theory, exact learning, property testing, coding theory, and computational complexity — each define a natural parameter governing their central questions: the minimum number of examples needed to teach a concept, the query complexity of identifying a function, the number of positions to check in a codeword, the certificate size for semantic uniqueness. Some pairwise connections among these quantities are known, and intellectual honesty requires a precise accounting. The relationship between teaching dimension and query complexity has been studied since the 1990s (Goldman and Mathias, 1996; Angluin, 1988). Identity testing complexity is closely related to query complexity in certain oracle models (Blais et al., 2012). The connection between local testability and identity testing has been explored in the coding theory literature (Goldreich and Sudan, 2006). In the five-node graph of identifications, at least three edges — TD query complexity, query complexity identity testing, and identity testing local testability — have precedents in the literature, though the exact formulations and settings differ. What appears to be new is: (a) the identification of all five with a concrete software engineering quantity — the minimum number of tests to achieve mutation score 1 — which provides the common vertex through which the entire graph connects; (b) the SpecP parameter, which is introduced here; and (c) the mutation symmetry group G_μ as the unifying algebraic mechanism that determines the quantitative regime across all five characterizations simultaneously. This identification is not merely formal bookkeeping; it means that the substantial body of results in each field transfers directly to mutation testing and vice versa, mediated by the algebraic structure of the mutation symmetry group.

The teaching dimension embedding (Theorem 2.7) established the first of these identifications. In this section we prove the remaining four, mediated by the algebraic structure of the *mutation symmetry group* G_μ . The unifying mechanism is that all five quantities are counting the same combinatorial object — a minimum set of constraints that pins down a function’s

identity in a space of alternatives — and the mutation system provides the common framework in which this identity becomes visible.

4.1 Mutation symmetry theory

The *mutation symmetry group* G_μ is the group of permutations on the input domain D that commute with the mutation operator: $\pi \in G_\mu$ if and only if $\mu(\pi \cdot P) = \pi \cdot \mu(P)$ for all programs P , where π acts on programs by precomposition. The structure of G_μ determines the *regime* of the mutation system.

Theorem 4.1 (Symmetry determines regime). Let $|G_\mu|$ denote the order of the mutation symmetry group. If $|G_\mu| \geq |D|!/\text{poly}(|D|)$ (the group is “large”), the mutation system is in Regime A: $\sigma(P, \mu) = O(\text{poly}(|D|))$ for all P . If $|G_\mu| \leq \text{poly}(|D|)$ (the group is “small”), there exist programs in Regime B with $\sigma(P, \mu) = \Omega(2^{|D|}/|D|)$. (*Lean: T6_15_symmetry_determines_regime.*)

The proof constructs two concrete mutation systems with identical mutant counts but opposite regimes. The *gate-swap system* (which permutes internal gates of a circuit) has a large symmetry group and yields polynomial σ ; the *input-flip system* (which flips individual input bits) has a small symmetry group and yields exponential σ . The mutant count is identical in both cases — only the symmetry differs.

This result answers a natural question: what determines whether a program is easy or hard to specify? The answer is not the number of mutants, nor the size of the program, nor the complexity of the function it computes. It is the symmetry of the mutation operator. When the mutation operator commutes with many permutations of the input domain, tests are powerful — each test simultaneously constrains the program’s behavior across all inputs related by those permutations. When the mutation operator commutes with few permutations, tests are local — each test constrains only a single point, and complete specification requires visiting nearly every input independently. Symmetry is what converts local observations into global constraints, and its absence is what makes specification expensive.

4.2 Learning theory: Teaching dimension

Theorem 4.2 (Teaching dimension identification). For programs over finite domains with faithful oracles, $\sigma(P, \mu)$ equals the teaching dimension of P ’s semantic equivalence class in the concept space induced by μ . (*Lean: T5_17_teaching_dimension_lower_bound, T5_17_teaching_dimension_upper_bound, T6_8_TD_eq_query_complexity.*)

The identification is exact, not merely an inequality. A minimum teaching set for P ’s equivalence class is precisely a minimum test suite achieving $\text{SC} = 1$, and vice versa. The proof constructs a bijection between teaching sets and specification-complete test suites that preserves cardinality.

4.3 Exact learning: Query complexity

Theorem 4.3 (Exact learning identification). Greedy mutation testing is an Angluin-style exact learner (Angluin, 1988) whose query complexity equals $\sigma(P, \mu)$ up to the set cover logarithmic factor. (*Lean: T6_6_greedy_is_exact_learner, T6_7_greedy_near_optimal.*)

The equivalence and membership queries of Angluin’s model correspond precisely to mutation-level and input-level testing: an equivalence query asks “does the candidate program match the target on all inputs?”, which is the question answered by running the full test suite against a mutant; a membership query asks “what does the program do on input x ?”, which is a single

test. The greedy algorithm of Section 3, viewed through this lens, is performing exact learning — it simply did not know it.

This identification has a consequence that extends beyond formalism. György et al. (2025) recently argued that exact learning — learning a function’s identity precisely, not merely approximately — is the appropriate framework for general intelligence, and that the gap between statistical and exact learning is where the hard problems live. Specification complexity provides a formal measure of exactly that gap. A program with $\sigma(P, \mu) = 1$ is one that a single example suffices to identify exactly — the statistical and exact learning tasks coincide. A program with $\sigma(P, \mu) = k$ is one where approximate identification (statistical learning) may be easy but exact identification requires k constraints. The specification complexity *is* the quantitative distance between approximately knowing what a program does and knowing exactly. The machinery of Sections 2–3 — the Blum axioms, the exponential separation, the dynamics, the phase transition — can be read, from this vantage point, as a theory of how hard exact identification is for a particular function class, parameterized by the mutation policy that defines “exactness.” Whether this reading extends to a formal reduction between specification complexity and exact learning in the sense of György et al. is a conjecture we regard as promising but do not claim to have established.

4.4 Property testing: Identity testing

Theorem 4.4 (Identity testing identification). $\sigma(P, \mu)$ equals the deterministic query complexity of identity testing under the mutation-induced distance: the minimum number of queries needed to distinguish P from every non-equivalent program in $\text{Mut}_\mu(P)$. (*Lean: T6_17_kappa_is_identity_testing_complexity.*)

The connection to property testing (Goldreich et al., 1998; Blais et al., 2012) yields a *testability dichotomy* that parallels the regime analysis: in Regime A, identity testing requires polylogarithmically many queries; in Regime B, it requires polynomially or exponentially many. (*Lean: T6_18_testability_dichotomy.*)

4.5 Coding theory: Local testability

Theorem 4.5 (Local testability identification). In the coding-theoretic interpretation where programs are codewords and mutations are errors, $\sigma(P, \mu)$ is the local testability parameter — the minimum number of positions that must be checked to verify codeword identity. (*Lean: T6_19_20_testabilityDependsOnCodeword, T6_19_20_P1_kappa, T6_19_20_P2_kappa.*)

An important subtlety emerges: σ depends on the *codeword*, not only the *message*. Two programs computing the same function can have different specification complexities if they are written differently, because the mutation operator acts on syntax. This is Theorem 2.3 (representation independence) viewed from the coding-theoretic side: σ is invariant under mutation-preserving transformations, but not under arbitrary rewrites.

4.6 Computational complexity: SpecP

Theorem 4.6 (SpecP). The class SpecP, consisting of all programs with $\sigma(P, \mu) = O(\text{poly}(|D|))$, contains P/poly in Regime A. The relationship between SpecP and standard complexity classes in Regime B is open. (*Lean: T6_22_specComplexity, T6_22_verificationNotReconstruction, T6_24_regimeAYes.*)

SpecP is the natural complexity class for specification: the programs whose computational identity can be pinned down by polynomially many constraints. The class is defined by a verification condition (a polynomial-size test suite that achieves $\text{SC} = 1$), not by a computational condition (a polynomial-time algorithm). This distinction is the specification analog of the P

vs. NP distinction: SpecP captures programs that are easy to *verify* (in the specification sense), regardless of whether they are easy to *compute*. The open question of SpecP’s relationship to standard classes in Regime B is, in this light, a question about whether there exist functions that are polynomially hard to specify but polynomially easy to compute — a specification-theoretic version of the question that drives computational complexity theory.

The five identifications are summarized in the following table:

Field	Quantity	Reference
Learning theory	Teaching dimension	Goldman and Mathias (1996)
Exact learning	Query complexity	Angluin (1988)
Property testing	Identity testing queries	Blais et al. (2012)
Coding theory	Local testability parameter	Goldreich and Sudan (2006)
Complexity theory	SpecP membership parameter	This paper

These identifications are equalities, not analogies — they are equalities of the same underlying combinatorial quantity, viewed through five different lenses. A theorem about teaching dimension is, by transport along the identification, a theorem about query complexity, identity testing, local testability, and SpecP membership. (We emphasize: this transfer is exact for finite-domain programs with faithful oracles. Whether it extends to broader settings is an open question.) The mutation symmetry group G_μ determines which regime each lens operates in and, consequently, the quantitative behavior of all five measures. This means that the regime analysis of Section 4.1 — the structural result that symmetry determines difficulty — applies uniformly across all five fields: the same algebraic condition that makes a program easy to specify makes it easy to teach, easy to identify, easy to test locally, and easy to certify. Within the scope of finite-domain programs, the fields are studying the same phenomenon from different vantage points.

5 Compositional Specification Theory

The composition gap theorem (Theorem 3.16) establishes how specification complexity behaves under pairwise composition. But real software systems are not pairs of functions — they are decomposition lattices with hundreds of modules, complex dependency graphs, and interfaces that may or may not respect the independence conditions under which γ vanishes. This section extends the pairwise theory to arbitrary modular decompositions, establishing the conditions under which local specifications compose into global ones and quantifying the cost when those conditions fail.

5.1 The sheaf condition

The key question for modular specification is: when do local specifications — test suites that achieve $SC = 1$ for individual modules — compose into a global specification that achieves $SC = 1$ for the entire system? The answer is governed by a compatibility condition on module interfaces.

Definition 5.1 (Sheaf condition). *A modular decomposition satisfies the sheaf condition if all declared module interfaces are compatible: local test suites for adjacent modules agree on the behavior of shared interface points.* (Lean: `sheafCondition`.)

Theorem 5.2 (Additive composition under sheaf condition). When the sheaf condition holds, local test suites compose additively:

$$\sigma_{\text{global}} \leq \sum_{m \in \text{modules}} \sigma_{\text{local}}(m).$$

No interface penalties are incurred. (*Lean: T7_17_sheaf_condition.*)

Theorem 5.3 (Sheaf failure bound). When the sheaf condition fails, the interface penalties are bounded:

$$\sigma_{\text{global}} \leq \sum_{m \in \text{modules}} \sigma_{\text{local}}(m) + \sum_{(i,j) \in \text{interfaces}} \gamma(i,j).$$

(*Lean: T7_17_sheaf_failure_bound.*)

The sheaf condition formalizes the intuition that well-designed module interfaces enable independent specification. When modules interact only through well-defined interfaces and those interfaces are compatible, the specification effort for the system is simply the sum of the specification efforts for its parts. The sheaf terminology is deliberate: the local-to-global principle here is the same one that governs sheaves in algebraic geometry — local data that agree on overlaps glue into global data.

5.2 The obstruction class

When the sheaf condition fails, the cost of failure is measured by the *obstruction class*.

Definition 5.4 (Obstruction class). The *obstruction class* of a modular decomposition is the total interface gap cost:

$$\mathcal{O} = \sum_{(i,j) \in \text{interfaces}} \gamma(i,j).$$

(*Lean: obstructionClass.*)

Theorem 5.5 (Obstruction characterization). The obstruction class vanishes if and only if every interface penalty is zero — equivalently, if and only if the sheaf condition holds:

$$\mathcal{O} = 0 \iff \forall (i,j) \in \text{interfaces}, \gamma(i,j) = 0.$$

(*Lean: T7_18_obstruction_zero_iff, T7_18_obstruction_zero_iff_sheaf.*)

The obstruction class measures “structural incompleteness” of a decomposition — the degree to which the modules’ interfaces fail to support independent specification. A decomposition with $\mathcal{O} = 0$ is *specification-clean*: local testing suffices. A decomposition with large $\mathcal{O}$ is *specification-dirty*: the modules are entangled through their interfaces, and additional specification effort proportional to $\mathcal{O}$ is required.

The decomposition objective $\sigma_{\text{local}} + \mathcal{O}$ provides a quantitative criterion for decomposition quality that extends the composition gap of Theorem 3.16 from pairs to arbitrary module graphs. Minimizing $\mathcal{O}$ over all decompositions of a program yields the optimal modular structure for specification — the decomposition that minimizes total specification effort. This is not a heuristic; it is a formally well-defined optimization problem with a clear objective function.

5.3 Distributed specification convergence

When multiple agents construct test suites concurrently — as in a distributed development team, a CI/CD pipeline with parallel test runners, or a multi-agent AI system — the convergence analysis of Section 3 must account for information sharing. The following results establish that distributed specification converges under surprisingly weak synchronization requirements.

Theorem 5.6 (Asynchronous specification convergence). If the sheaf condition holds and every module is covered by at least one agent’s specification effort, then global specification completeness is achieved: local coverage under sheaf gluing implies global SC = 1. (*Lean: T7_19_asynchronous_specification.*)

The key insight is that the sheaf condition provides exactly the gluing mechanism needed for distributed convergence. Each agent specifies its assigned modules locally; the sheaf condition guarantees that these local specifications are compatible and compose into a global specification without coordination overhead.

Theorem 5.7 (Convergence under partial communication). *Distributed specification converges even under asynchronous, lossy communication, provided two invariants hold:*

- (i) *Monotonicity: the global killed set grows monotonically: $K_i \subseteq K_{i+1}$.*
- (ii) *Communication adequacy: every interface mutant is eventually observed by some agent.*

These two conditions suffice for eventual full specification completeness. (Lean: `T7_20_convergence_under_partial_communication`.)

Theorem 5.7 identifies exactly the two synchronization primitives needed for distributed convergence: partial monotonicity (which ensures progress and tolerates message reordering) and eventual visibility (which ensures completeness and tolerates intermittent communication failures). The proof uses a fixed-point argument: monotonicity guarantees that progress is never lost, and eventual visibility guarantees that every remaining mutant is eventually addressed.

These results have immediate practical implications. Monotonicity is automatically satisfied by any testing process that does not discard passing tests — the killed set can only grow. Communication adequacy requires only that interface-crossing tests are eventually shared between teams — not that they are shared immediately or reliably. The conditions are deliberately weak: they tolerate message loss, reordering, and arbitrary delays. The only hard requirement is that interface information *eventually* propagates, which is satisfied by any reasonable CI/CD system with shared test result databases.

6 Implementation: LintGate

The theory of Sections 2–5 defines quantities — σ , the specification trajectory, the phase transition point, the regime classification, the composition gap, the obstruction class — that are in principle computable for real programs. LintGate is an open-source mutation analysis tool designed to operationalize this framework for working software engineers. We describe its architecture and the theoretical predictions it is designed to test; empirical evaluation on production codebases is ongoing and will be reported separately.

6.1 Architecture

LintGate implements 84 MCP (Model Context Protocol) tools organized around the specification complexity framework. The core mutation analysis pipeline generates mutants, tracks specification trajectories, computes regime estimates, and produces decomposition recommendations based on the phase transition analysis of Section 3.3. The tool replaces traditional mutation score reporting (a single number between 0 and 1) with trajectory-aware reporting: the specification score over time, the estimated σ , the detected regime, and the transition point i^* .

6.2 Theoretical predictions for empirical validation

The theory makes several falsifiable predictions that LintGate is designed to test. First, the exponential decay of Theorem 3.2 predicts that specification trajectories on real programs should exhibit rapid initial progress (the bulk phase) followed by a transition to slower, targeted testing (the tail phase), with the transition point determined by σ . Second, the decomposition theory predicts that extracting independent concerns from a function with $\sigma = k_1 \cdot k_2$ into two functions

with $\sigma = k_1$ and $\sigma = k_2$ should reduce the total specification effort from $O(k_1 k_2 \ln n)$ to $O((k_1 + k_2) \ln n)$. Third, the regime analysis (Theorem 4.1) predicts that mutation systems with high symmetry should exhibit polynomial σ while those with low symmetry should exhibit exponential σ , with observable consequences for whether random or greedy testing strategies are more efficient. Fourth, the sheaf condition (Theorem 5.2) predicts that well-decomposed systems with clean interfaces should have near-additive total specification complexity, while monolithic systems with entangled interfaces should exhibit superadditive σ .

6.3 Decomposition prescriptions

The operational chain from theory to practice is: mutation pressure reveals specification gaps; specification gaps identify entangled concerns; decomposition along entangled concerns produces components whose behavioral dimensions are independent and whose specification complexity is therefore additive. Surviving mutant categories map to behavioral degrees of freedom, and LintGate classifies survivors by mutation type (arithmetic, relational, logical, boundary) to identify which categories dominate the tail phase.

The composition gap theorem (Theorem 3.16) provides formal backing for this operational chain: when decomposition reduces the interface mutant count, $\gamma(A, B)$ decreases, and the specification effort shifts from multiplicative to additive. The interface mutant bound (Theorem 3.17) makes the chain quantitatively actionable — LintGate can estimate γ directly from surviving mutant analysis, providing a computable criterion for when decomposition will reduce specification effort. The obstruction class (Theorem 5.5) extends this to modular decompositions: LintGate can estimate $\mathcal{O}$ across a module graph and identify the interfaces whose penalties dominate the total specification cost. Whether this reduction is observable on production codebases, and whether the decomposition recommendations are actionable in practice, requires systematic empirical study; evaluation is ongoing and will be reported separately.

7 Methodology: AI-Assisted Formalization

The theoretical results in this paper are formalized in approximately 10,000 lines of Lean 4 with Mathlib, organized across seven phases. This formalization serves a dual purpose: it provides machine-checked verification of the mathematical claims, and it serves as a case study in a particular mode of research — one in which the human contribution is structural insight and the machine contribution is formal translation. We describe the methodology here, both for transparency and because the workflow itself raises questions about where the intellectual bottleneck lies in cross-disciplinary research.

The division of labor was as follows. The author identified the structural connections between mutation testing, specification complexity, and the five fields of Section 4. This involved recognizing that the minimum test suite for $SC = 1$ is the same combinatorial object as a minimum teaching set, an optimal query strategy for identity testing, and so forth. The author also formulated the theorem statements and proof strategies informally.

An AI theorem prover (Aristotle; Harmonic, 2025) translated the informal proof strategies into Lean 4 with Mathlib. This translation is nontrivial — the gap between an informal proof sketch and a machine-checked proof involves hundreds of lemmas, type coercions, decidability instances, and tactic choices that the informal sketch does not specify. The AI system handled this translation; the author did not write Lean code directly.

Lean 4’s type checker verified all proofs. This is the critical point: the correctness of the formalization depends on Lean’s kernel, not on the AI system. If the AI theorem prover produced an incorrect proof, Lean would reject it. The trust chain is: author (structural insights) $\rightarrow$ AI

(formal translation) $\rightarrow$ Lean (verification). The middle link is untrusted — it is the outer links that bear the epistemic weight.

The formalization uncovered several errors in the author’s informal reasoning, primarily involving edge cases in the finiteness and decidability arguments. These were corrected during the formalization process. We regard this as a feature of the methodology: the formalization serves as a rigorous check on the informal theory, catching errors that peer review might miss.

The formalization evolved through seven phases, and the evolution itself is informative. The first four phases comprised approximately 2,000 lines of foundational definitions and basic properties. Phases 5 and 6 — the specification complexity theory and the five-field identification — required approximately 3,800 lines and constituted the bulk of the mathematical content. Phase 7 — the specification dynamics and compositional theory — required approximately 3,500 lines and produced theorems that unified the earlier phases, suggesting that the theory was converging on its natural formulation. The total of approximately 10,000 lines is modest by Lean 4 standards, consistent with a theory whose power comes from choosing the right definitions rather than from technically elaborate proofs. The hardest part of the formalization was not any individual proof but the recognition, during Phase 6, that the five-field identification was exact rather than merely analogical — that the witnessing structures were not similar but identical. This recognition required no Lean; it required staring at the definitions until the isomorphism became obvious.

We note that this workflow — human insight, AI formalization, machine verification — is not specific to this paper. It suggests a mode of mathematical research in which the bottleneck shifts from proof construction to theorem identification. Whether this is desirable is a question we leave to the reader.

8 Related Work

8.1 Foundational mutation testing

DeMillo, Lipton, and Sayward (1978) and Hamlet (1977)¹ introduced mutation testing as a method for evaluating test suite adequacy, grounded in the *competent programmer hypothesis*: that programmers write programs close to correct, so small syntactic perturbations are a reasonable model of likely faults. The field has developed extensively; Jia and Harman (2011) and Papadakis et al. (2019) provide comprehensive surveys. The equivalent mutant problem — that some mutants are semantically identical to the original — was identified by Budd and Angluin (1982) and remains a central challenge. Morris (1968) established observational equivalence in denotational semantics, providing the formal backdrop against which mutation-based distinguishability is defined: two programs are equivalent if and only if no observation can distinguish them, and a surviving mutant witnesses the failure of the test suite to make a particular observation.

The reframing of mutation score as specification completeness is, to our knowledge, new, as is the definition of specification complexity as a program property rather than a test suite property. The algebraic specification tradition (Goguen et al., 1978) — in which a program’s meaning is determined by the equations it satisfies — provides a natural precursor: our test suites are finite sets of input-output equations, and specification completeness means that no non-equivalent program satisfies the same equations. The connection between mutation testing and algebraic specification has not, to our knowledge, been previously formalized.

¹Not to be confused with the other Hamlet, whose problems with testing were of a different nature (Shakespeare, c. 1600).

8.2 Programs as specifications

The philosophical foundation of this work is Abelson and Sussman’s (1996) framing in *Structure and Interpretation of Computer Programs* that programs are “precise specifications of computational processes.” This is not a pedagogical simplification but an ontological claim: the program *is* the specification, and the question of what it computes is a question about the completeness of that specification relative to external constraints. Sussman’s concept of *abstraction barriers* — that well-designed programs separate concerns into layers that interact only through defined interfaces — maps directly to our compositional specification theory (Section 5): the sheaf condition (Theorem 5.2) formalizes exactly when abstraction barriers are respected, and the obstruction class (Theorem 5.5) measures the specification cost when they are violated. Sussman’s distinction between declarative knowledge (“what is”) and imperative knowledge (“how to”) has a natural interpretation in our framework: a fully specified program is one whose declarative content — the function it computes — is completely determined by its test suite, regardless of its imperative implementation.

Claessen and Hughes (2000) introduced property-based testing with QuickCheck, establishing that programs can be specified by properties rather than individual test cases. The relationship between property-based testing and mutation testing has been explored informally but not, to our knowledge, formalized. Our framework provides a bridge: a property that holds for the program but fails for a mutant is a specification constraint that kills that mutant, and the minimum number of such properties needed for complete specification is a natural variant of σ over property-based test suites.

8.3 Specification completeness as measurable property

Three independent research programs have recently converged on aspects of the theory developed here, each addressing specification completeness from a different angle. Their synthesis — connecting specification measurement, efficient execution, and test synthesis into a unified complexity-theoretic framework — is a central contribution of this paper.

Xu et al. (Waterloo). FAST (Ji and Xu, 2023) applies mutation testing to formally verified systems (seL4, CompCert), mutating code and checking whether the formal specification catches the mutation. This is precisely the operation our framework performs with test suites as implicit specifications: where FAST checks satisfiability of formal specs, we check test suite distinguishability. FAST formalizes “specification blind spots” as gaps where mutated code still conforms to the spec — each blind spot is a behavioral degree of freedom in our terminology. Xu (2024) provides a taxonomy for specification quality in “Not All Move Specifications Are Created Equal,” establishing that specifications vary in their constraining power — some are tautological (structural assertions that pass regardless of behavior) while others are substantive (value assertions that pin down specific outputs). This taxonomy maps to the specification lattice implicit in our framework and motivates the distinction between crash-kills and assertion-kills in our oracle model. eBPF SpecCheck (Lyu et al., 2025) formalizes this distinction as an oracle taxonomy: crash oracles prove crash-freedom but not behavioral correctness; specification oracles prove the function computes the right output. FuzzSlice (Xu, 2024b) demonstrates static analysis validated by targeted execution — the pattern of using cheap static signals to guide expensive dynamic verification that our decomposition prescriptions (Section 6) operationalize.

Lévai and McMinn (Sheffield). mutest-rs (Lévai and McMinn, 2023; 2026) demonstrates that mutation testing can be made efficient enough for routine use through compiler-integrated mutation via static dispatch tables — all mutations compiled into one binary, selected at runtime by a global handle — and non-conflicting mutation batching, in which mutations targeting functions with disjoint call graphs execute simultaneously. Their formalization (ICST 2023, extended in TOSEM 2026) proves that batching achieves 66.4% runtime reduction and provides a mathematical cost model for mutation execution. The efficiency results are essential for

practical specification complexity measurement: without them, computing σ for real programs would be prohibitively expensive. Sheffield’s work provides the execution architecture; our contribution is the specification-level interpretation and the complexity-theoretic framework that gives the measurements meaning.

Vikram and Padhye (CMU). *mu2* (Vikram and Padhye, 2023) inverts the standard mutation testing relationship: instead of running tests against mutants, it uses mutation kill count as a fitness function for a greybox fuzzer. Inputs that kill more mutants are prioritized; inputs that kill none are discarded. This is, in our framework, an adaptive algorithm for constructing specification-complete test suites guided by the greedy criterion of Theorem 3.1 — each selected input maximizes the kill increment ΔK_i , which is precisely the greedy set cover strategy whose convergence we prove in Theorem 3.2. *mu2*’s cartography-then-exploit architecture (map mutation sites once, amortize cost across generated inputs) and dead mutant retirement (killed mutants excluded from future runs, surviving set shrinks monotonically) are operational implementations of specification trajectory construction. The connection between *mu2*’s empirical success and our convergence guarantees is, we believe, not coincidental: *mu2* works because the specification process has favorable greedy dynamics, and our theory explains why.

The synthesis. These three research programs address different aspects of the same phenomenon. Xu shows that specifications have measurable completeness and quality taxonomies. Sheffield shows how to measure efficiently. Padhye shows how to close the loop between measurement and synthesis. Our contribution is the complexity-theoretic framework that connects these operational results: the Blum axiom satisfaction that legitimizes σ as a complexity measure, the five-field identification that reveals specification complexity as a known quantity in disguise, the dynamics that explain convergence, and the compositional theory that extends from individual programs to modular systems.

8.4 Learning theory and property testing

Teaching dimension. Goldman and Mathias (1996) introduced the teaching dimension as the minimum number of labeled examples needed to uniquely identify a concept. The connection between teaching dimension and mutation testing has not, to our knowledge, been previously observed. Our identification (Theorem 4.2) is exact, not asymptotic.

Exact learning. Angluin (1988) introduced the model of exact learning with equivalence and membership queries. Budd and Angluin (1982) studied the equivalent mutant problem, which is the mutation-theoretic manifestation of the fundamental difficulty in exact learning: distinguishing the target from all alternatives requires solving an equivalence problem that may be undecidable in general. György et al. (2025) recently argued that exact learning, rather than statistical learning, is the appropriate framework for general intelligence, and that the gap between statistical and exact learning — what they term the “statistical-to-exact learning gap” — is where the genuinely hard problems of intelligence reside. Our results provide a formal framework for this thesis. Specification complexity $\sigma(P, \mu)$ is a precise measure of the distance between statistical and exact identification of a function: the number of additional constraints required to move from approximate knowledge to complete specification. The exponential separation (Theorem 2.6) shows that this gap can be arbitrarily large; the dynamics (Section 3) characterize how the gap closes under greedy querying; the regime analysis (Theorem 4.1) identifies the algebraic condition — mutation symmetry — that determines whether the gap is polynomial or exponential. The five-field identification (Section 4) shows that this gap is the same quantity that learning theory, property testing, and coding theory have been studying independently.

Property testing. Goldreich, Goldwasser, and Ron (1998) initiated the study of property testing; Blais, Brody, and Matulef (2012) developed the theory of property testing for Boolean functions. Our identification of σ with the query complexity of identity testing (Theorem 4.4)

connects mutation testing to this literature. The testability dichotomy (Regime A vs. Regime B) parallels known dichotomies in property testing.

Locally testable codes. Goldreich and Sudan (2006) defined locally testable codes and studied their parameters. Our Theorem 4.5 identifies σ as the local testability parameter in the coding-theoretic interpretation of programs.

8.5 Compositionality and sheaf theory

The use of sheaf-theoretic language for compositionality has precedents in database theory (Abramsky and Brandenburger, 2011), contextuality in quantum mechanics (Abramsky and Brandenburger, 2011), and network coding (Ghrist, 2007). Our application to specification compositionality appears to be new: the sheaf condition (Theorem 5.2) provides a formal criterion for when local specifications glue into global ones, with the obstruction class (Theorem 5.5) quantifying the cost of failure. The ADJ group (Goguen et al., 1978) established the algebraic approach to software specification, in which compositionality is a first-class concern; our sheaf condition can be read as a mutation-theoretic criterion for when the ADJ group’s compositional ideal is achievable.

8.6 AI-assisted mathematics

Tao (2024) discussed the use of AI tools in mathematical research. The AlphaProof system (Silver et al., 2024) demonstrated AI-assisted theorem proving at the International Mathematical Olympiad level. Our methodology differs in that the AI system handles formalization rather than discovery; the structural insights are the author’s contribution.

9 Future Directions

The framework developed in this paper addresses composition for arbitrary module decompositions (Section 5) and establishes convergence for distributed specification (Theorem 5.7). Several natural extensions remain.

Topological invariants. The mutation neighborhood complex (the simplicial complex whose simplices are sets of mutants killed by a single test) may carry topological information about specification difficulty. Whether σ is determined by the homology of this complex is an open conjecture. Preliminary work (T7.21–T7.23 in the formalization, not yet complete) suggests that a topological frontier theory could provide invariants that are coarser than σ but more efficiently computable.

Tightening the approximation ratio. The $O(\ln n)$ gap between the greedy algorithm and the optimum (Theorem 3.3) is inherited from generic set cover. The structure of mutation systems may admit tighter bounds; closing the gap, or proving it is tight, would resolve an analogue of the set cover hardness question in the specification setting.

SpecP and standard complexity classes. The relationship between SpecP and standard complexity classes in Regime B remains open. In particular: is there a function that is easy to compute (in P) but hard to specify ($\sigma = \Omega(2^n/n)$)? A positive answer would establish a specification-theoretic separation analogous to (but independent of) the P vs. NP question. A negative answer would imply that computational efficiency somehow constrains specification difficulty — a deep structural claim about the relationship between computing a function and verifying its identity.

Specification complexity and exact learning theory. The identification of σ with exact learning query complexity (Theorem 4.3) suggests a broader program: use the specification complexity machinery to answer open questions in exact learning. In particular, the teaching dimension is

known to be bounded above by $O(\text{poly}(\text{VC}))$ for concept classes with bounded displacement (a result formalized in Phase 6 of our Lean development). Whether this bound extends to broader mutation classes, and whether the exponential separation of Theorem 2.6 has implications for the statistical-to-exact learning gap identified by György et al. (2025), are open questions with implications beyond software engineering.

Canonical mutation policies. A satisfactory theory of mutation policies would include: a taxonomy of natural policies, conditions under which σ is invariant across a class of policies, and an analogue of the polynomial simulation theorem from circuit complexity. We regard this as the most important open problem in the framework.

10 Conclusion

Mutation testing, for nearly fifty years, has been understood as measuring test suite quality. We have argued that it is more naturally understood as measuring specification completeness, and that this change in perspective — from a quality metric to a complexity measure — is not merely terminological but structural.

The reframing reveals that specification complexity $\sigma(P, \mu)$ is a legitimate complexity measure satisfying the Blum axioms — straightforwardly for finite domains, and substantively for partial programs over infinite domains via an obstruction package construction. It admits an exponential separation between program classes: not between programs that are easy and hard to compute, but between those that are easy and hard to specify. It exhibits a characteristic dynamics with exponential convergence, phase transitions, and information-theoretic optimality guarantees that connect the abstract measure to observable testing phenomena. And it equals established quantities across five fields of theoretical computer science — learning theory, exact learning, property testing, coding theory, and computational complexity — each of which independently discovered the same underlying combinatorial object.

The compositional specification theory extends these results from individual programs to modular software systems. The composition gap theorem quantifies the specification cost of component interaction; the sheaf condition identifies when local specifications compose without penalty; the obstruction class measures the total cost of violating the sheaf condition; and the distributed convergence theorems guarantee that concurrent specification efforts converge under weak synchronization conditions. Together, these results provide the formal bridge from “specification complexity is a program property” to “here’s how decomposition affects it quantitatively” — the theory-to-practice connection that makes the framework actionable for software engineering.

The five-field identification is, in our view, the central result — not because the individual connections are all new (some pairwise relationships are known), but because their simultaneous realization in a concrete software engineering quantity suggests that specification complexity is measuring something natural. The mutation symmetry group G_μ determines which quantitative regime all five characterizations operate in, providing a structural explanation for why some programs are easy and others are hard to test, to teach, to identify, to certify, and to verify.

The connection to exact learning is suggestive and, if it can be made fully rigorous, potentially the most consequential aspect of the framework. György et al. (2025) argue that the statistical-to-exact learning gap is where the hard problems of intelligence reside. Specification complexity provides a candidate formal measure of that gap, parameterized by the mutation policy that defines what “exact” means. The exponential separation (Theorem 2.6) shows the gap can be arbitrarily large. The phase transition (Theorem 3.4) shows where the problem shifts from statistical to exact. The mutation symmetry group (Theorem 4.1) determines whether the gap is polynomial or exponential. Whether these results constitute a theory of the difficulty

of exact identification, or merely a suggestive analogy, depends on establishing a formal reduction between their function class model and our mutation system model — a question we leave open, noting that the framework was discovered through the seemingly unrelated project of understanding what mutation testing is really measuring, which is at minimum circumstantial evidence for its naturalness.

The theory is machine-checked in approximately 10,000 lines of Lean 4, implemented in a working tool, and open-source. The formalization required no trust in the AI system that produced it — only in the Lean type checker that verified it. Whether this workflow — human structural insight, AI formalization, machine verification — represents a new mode of mathematical research or merely an efficient use of existing tools is a question whose answer, we suspect, will become clearer with time. What is clear already is that the bottleneck was not proof construction; it was recognizing that five fields were studying the same object.

11 References

Abelson, H. and Sussman, G. J. (1996). *Structure and Interpretation of Computer Programs*, 2nd ed. MIT Press.

Abramsky, S. and Brandenburger, A. (2011). The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics*, 13(11), 113036.

Angluin, D. (1988). Queries and concept learning. *Machine Learning*, 2(4), 319–342.

Blais, E., Brody, J., and Matulef, K. (2012). Property testing lower bounds via communication complexity. *Computational Complexity*, 21(2), 311–358.

Blum, M. (1967). A machine-independent theory of the complexity of recursive functions. *Journal of the ACM*, 14(2), 322–336.

Budd, T. A. and Angluin, D. (1982). Two notions of correctness and their relation to testing. *Acta Informatica*, 18(1), 31–45.

Chvátal, V. (1979). A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3), 233–235.

Claessen, K. and Hughes, J. (2000). QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 268–279.

DeMillo, R. A., Lipton, R. J., and Sayward, F. G. (1978). Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4), 34–41.

Ghrist, R. (2007). Barcodes: The persistent topology of data. *Bulletin of the American Mathematical Society*, 45(1), 61–75.

Goguen, J. A., Thatcher, J. W., and Wagner, E. G. (1978). An initial algebra approach to the specification, correctness, and implementation of abstract data types. In *Current Trends in Programming Methodology*, Vol. 4, Prentice-Hall, 80–149.

Goldreich, O., Goldwasser, S., and Ron, D. (1998). Property testing and its connection to learning and approximation. *Journal of the ACM*, 45(4), 653–750.

Goldreich, O. and Sudan, M. (2006). Locally testable codes and PCPs of almost-linear length. *Journal of the ACM*, 53(4), 558–655.

Goldman, S. A. and Mathias, H. D. (1996). Teaching a smarter learner. *Journal of Computer and System Sciences*, 52(2), 255–267.

György, A., Lattimore, T., Lazić, N., and Szepesvári, C. (2025). Beyond statistical learning: Exact learning is essential for general intelligence. *arXiv preprint arXiv:2506.23908*.

Hamlet, R. G. (1977). Testing programs with the aid of a compiler. *IEEE Transactions on Software Engineering*, SE-3(4), 279–290.

- Ji, Y. and Xu, H. (2023). FAST: FPGA-based Accelerated Software Testing. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- Jia, Y. and Harman, M. (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5), 649–678.
- Lévai, T. and McMinn, P. (2023). mutest-rs: A compiler-integrated mutation testing framework for Rust. In *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST)*.
- Lévai, T. and McMinn, P. (2026). Non-conflicting mutation batching: A mathematical cost model for efficient mutation testing. *ACM Transactions on Software Engineering and Methodology (TOSEM)*.
- Lyu, S., Payer, M., Xu, H., et al. (2025). eBPF SpecCheck: Specification-based oracle taxonomy for eBPF verification. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*.
- Morris, J. H. (1968). *Lambda-calculus models of programming languages*. PhD thesis, MIT.
- Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., and Harman, M. (2019). Mutation testing advances: An analysis and survey. *Advances in Computers*, 112, 275–378.
- Shannon, C. E. (1949). The synthesis of two-terminal switching circuits. *Bell System Technical Journal*, 28(1), 59–98.
- Silver, D., et al. (2024). AlphaProof and AlphaGeometry 2. *Google DeepMind Technical Report*.
- Tao, T. (2024). Machine-assisted proof. *Notices of the AMS*, 71(7), 896–902.
- Vikram, A. and Padhye, R. (2023). mu2: Mutation-guided seed generation for greybox fuzzing. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*.
- Xu, H. (2024). Not all Move specifications are created equal. In *Proceedings of the IEEE Workshop on the Security of Blockchain and Decentralized Systems (LangSec)*.
- Xu, H. (2024b). FuzzSlice: Static analysis validated by targeted execution. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*.

A Lean 4 Formalization

The complete formalization is available at [repository URL]. It comprises seven phases:

Phase	Content	Lines	Key theorems
1–4	Core definitions (Program, Test, MutationSystem)	~2,000	Foundation
5	Specification complexity	~1,800	T5.1–T5.18
6	Exact learning and five-field identification	~2,000	T6.1–T6.24
7	Specification dynamics and compositional theory	~3,500	T7.0–T7.20

Build instructions: `lake build` in the repository root. All files compile with Lean 4.x.0 and Mathlib (commit hash [to be specified]).

B Theorem Index

Theorem	Statement	Lean reference
2.2	Finiteness	T5_1_spec_complexity_finite
2.3	Representation independence	T5_2_representation_bound

Theorem	Statement	Lean reference
2.4	Kolmogorov independence	T5_3_conditional_refactor
2.5	Blum axioms	T5_7b_decidable_finite_domain, T5_8_blum_axiom_1
2.6	VCC exponential separation	T5_11_Verification_Can_Be_ Exponentially_Easier_Than_ Computation
2.7	Teaching dimension embedding	T5_17_teaching_dimension_ lower_bound, T5_17_ teaching_dimension_upper_ bound
3.1	Proportional progress	greedy_proportional_progress
3.2	Exponential decay	exponential_decay_of_survivors
3.3	Greedy approximation ratio	T5_12_greedy_ approximation_ratio
3.4	Phase transition	phase_transition, transition_step_properties
3.5	Stability bound	greedy_stability_bound_final
3.6	Lyapunov monotonicity (remark)	lyapunov_monotonicity
3.7	Entropy monotonicity	entropy_monotonicity, entropy_properties
3.8	Information per test	greedy_information_content, total_information_content
3.9	Trajectory comparison	T7_7_trajectory_comparison
3.11	Free energy bound	specificationFreeEnergyBound
3.12	Redundancy characterization	redundantIffZeroInformation, greedy_redundancy_ratio_zero, redundancy_ratio_limit_eq_one
3.13	Spectrum evolution	spectrumSizeAntitone, spectrum_max_antitone, spectrum_min_monotone
3.14	Dynamic regime transition	dynamicRegimeTransition
3.15	Regime decomposition	regimeTransition, supports_Regime_A_mono
3.16	Composition gap	compositionGap, gammaNonneg, gammaZeroIfIndependent
3.17	Interface mutant bound	gammaLeInterfaceMutantsCard
4.1	Symmetry determines regime	T6_15_symmetry_determines_ regime
4.2	Teaching dimension identification	T5_17_teaching_dimension_ lower_bound, T5_17_ teaching_dimension_upper_ bound, T6_8_TD_eq_query_complexity
4.3	Exact learning identification	T6_6_greedy_is_exact_learner, T6_7_greedy_near_optimal
4.4	Identity testing identification	T6_17_kappa_is_identity_ testing_complexity
4.5	Local testability identification	T6_19_20_ testabilityDependsOnCodeword, T6_19_20_P1_kappa, T6_19_20_P2_kappa
4.6	SpecP	T6_22_specComplexity, T6_ 22_ verificationNotReconstruction, T6_24_regimeAYes

Theorem	Statement	Lean reference
5.2	Sheaf condition	<code>T7_17_sheaf_condition</code>
5.3	Sheaf failure bound	<code>T7_17_sheaf_failure_bound</code>
5.5	Obstruction characterization	<code>T7_18_obstruction_zero_iff,</code> <code>T7_18_obstruction_zero_</code> <code>iff_sheaf</code>
5.6	Asynchronous convergence	<code>T7_19_asynchronous_specification</code>
5.7	Partial communication convergence	<code>T7_20_convergence_under_</code> <code>partial_communication</code>

C LintGate

LintGate is available at [repository URL]. Installation: `pip install lintgate`. The mutation analysis pipeline is accessible via 84 MCP tools; see the repository documentation for usage. Empirical evaluation of the theoretical predictions described in Section 6.2 is in progress and will be reported in a companion paper.